# Programowanie obiektowe i inżynieria oprogramowania

Lab 5.2:

- poszukiwanie elementu w wektorze/tablicy,
- wartość domyślna argumentu funkcji.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

# Poszukiwanie elementu w wektorze/tablicy

Moja propozycja implementacji funkcji
dot. weryfikacji obecności cieczy w kubku

# Poszukiwanie elementu w wektorze/tablicy

Uogólnienie działania funkcji na dowolnym zbiorze typu vector<Tsubstance>

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | TCup | get_substance_id(std::vector< TSubstance

```
1  #pragma once
2  #include <iostream>
3  #include <vector>
4
5  #include "TSubstance.h"
6
7
8  class TCup
9  {
10     std::vector<TSubstance> substances;
11     std::vector<double> volumes;
12
13 public:
14     void add(TSubstance substance, double volume_in_ml);
15     void add(std::string name, double volume_in_ml);
16     void show();
17
18 private:
19     int get_substance_id(std::vector<TSubstance> substances_buffer, std::string name);
20 };
```

# Poszukiwanie elementu w wektorze/tablicy

JEST!

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | TCup | get_substance_id(vector<TSubstance> su

```
22
23  void TCup::add(string name, double volume_in_ml)
24  {
25      int _id = get_substance_id(substancje, name);
26
27      if (_id >= 0)
28      {
29          this->add(substancje[_id], volume_in_ml);
30      }
31  }

int TCup::get_substance_id(vector<TSubstance> substances_buffer, string name)
{
    int count = substances_buffer.size();


    int _id = -1;
    bool run = count;
    while (run)
    {
        _id += 1;
        string sub_name = substances_buffer[_id].get_name();

        if (sub_name == name) run = false;

        if (_id + 1 == count)
        {
            cout << "Nie znaleziono plynu o podanej nazwie: \"" << name << "\"!\n";
            run = false;
            _id = -1;
        }
    }
54
55
56      return _id;
57  }
```

Było…

```
20  int TCup::get_substance_id(string name)
21  {
22      int count = substancje.size();
23      int _id = -1;
24      bool run = true;
25
26      while (run)
27      {
28          _id += 1;
29          string sub_name = substancje[_id].get_name();
30
31          if (sub_name == name) run = false;
32
33          if (_id + 1 == count)
34          {
35              cout << "Nie znaleziono plynu o podanej na
36              run = false;
37              _id = -1;
38          }
39      }
40
41      return _id;
42  }
```

# Poszukiwanie elementu w wektorze/tablicy

## Modyfikacja pierwotnej funkcji add

TCup.h | TCup.cpp | TSubstance.cpp | TSubstance.h

kubki | TCup | get_substance_id(vector<TSubstan

```
#include "TCup.h"
using namespace std;

void TCup::add(TSubstance substance, double volume_in_ml)
{
    string new_name = substance.get_name();
    int substance_pos_in_cup = get_substance_id(this->substances, new_name);

    if (substance_pos_in_cup == -1)
    {

        substances.push_back(substance);
        volumes.push_back(volume_in_ml / 1e6);

    } else {

        this->volumes[substance_pos_in_cup] += volume_in_ml / 1e6;

    }

}
```

# Poszukiwanie elementu w wektorze/tablicy

kubki.cpp*

kubki (Globalny zasięg)

```cpp
// kubki.cpp : Ten plik zawiera funkc
//

#include <iostream>
#include <vector>
#include "TSubstance.h"
#include "TCup.h"
using namespace std;


int main()
{
    cout << "Hello World!\n";

    TCup kubek1;
    kubek1.add(substancje[0], 100);
    kubek1.add(substancje[1], 50);
    kubek1.add("woda", 30);
    kubek1.add("kwas", 1);
    kubek1.show();
}
```

Konsola debugowania programu Microsoft Visual Studio

```
Hello World!
Nie znaleziono plynu o podanej nazwie: "mleko"!
Nie znaleziono plynu o podanej nazwie: "kwas"!
0. woda; volume: 130 ml; mass: 129.714 g
1. mleko; volume: 50 ml; mass: 51.5 g
Volume coe: 72.2222 - 27.7778 -
Mass coe: 71.5806 - 28.4194 -
```

# Wartość domyślna argumentu funkcji

# Wartość domyślna argumentu funkcji

```
Konsola debugowania programu Microsoft Visual Studio
Hello World!
Nie znaleziono plynu o podanej nazwie: "mleko"!
Nie znaleziono plynu o podanej nazwie: "kwas"!
0. woda; volume: 130 ml; mass: 129.714 g
1. mleko; volume: 50 ml; mass: 51.5 g
Volume coe: 72.2222 - 27.7778 -
Mass coe: 71.5806 - 28.4194 -
```

Celem jest nie wyświetlanie informacji o braku mleka w kubku.

Funkcja poszukująca elementu w wektorze ma nas informować o braku cieczy jedynie w przypadku zmiennej globalnej, menu substancji.

W przypadku dodawania cieczy do kubka informacja ta jest zbędna – jeszcze jej nie ma w kubku (stąd alert), ale zaraz będzie.

Funkcja poszukująca elementu w wektorze jest już porządną funkcją z kawałkiem kodu. Nie ma sensu pisać jej kolejnej wersji. Poprzez dodatkowy argument funkcji wprowadzimy sobie dwa tryby jej działania – z komunikatem do konsoli i bez.

# Wartość domyślna argumentu funkcji

TCup.h | TSubstance.h

Kubeczki_zajecia1 | (Globalny zasięg)

```
1   #pragma once
2   #include <vector>
3
4   #include "TSubstance.h"
5
6   class TCup
7   {
8       std::vector<TSubstance> substances;
9       std::vector<double> volumes;
10
11  public:
12      void add(TSubstance substance, double volume_in_mm);
13      void add(std::string name, double volume_in_mm);
14      void show();
15
16  private:
17      void print_coe_vol();
18      void print_coe_mass();
19      int get_substance_id(std::vector<TSubstance> substances_buffer, std::string name, bool alert=true);
20  };
```

Ostatni argument służy do decydowania czy w tym konkretnym przypadku wywołania funkcji chcemy dostawać w konsoli informacje o braku substancji w menu. Domyślnie chcemy taką informacje dostawać, więc **ustawiamy wartość domyślną na *true***.

# Wartość domyślna argumentu funkcji

main.cpp | TSubstance.cpp | TCup.cpp

Kubeczki_zajecia1 | (Globalny zasięg)

```
87    }
88
89    int TCup::get_substance_id(vector<TSubstance> substances_buffer, string name, bool alert)
90    {
91        int count = substances_buffer.size();
92        int _id = -1;
93        bool run = count;
94
95        while (run)
96        {
97            _id += 1;
98            string name_temp = substances_buffer[_id].get_name();
99
100           if (name_temp == name) run = false;
101
102           if ((_id + 1 == count) and (run == true)
103           {
104               if (alert) cout << "Nie znaleziono plynu o podanej nazwie: " << name << "!\n";
105
106               run = false;
107               _id = -1;
108           }
109       }
110
111       return _id;
112   }
```

# Wartość domyślna argumentu funkcji

W przypadku dodawania substancji do kubka nie chcemy informacji o jej aktualnym braku w kubku, w trakcie jej dodawania...

main.cpp | TSubstance.cpp | TCup.cpp

Kubeczki_zajecia1 | (Globalny zasięg)

```cpp
#include "TCup.h"
using namespace std;

void TCup::add(TSubstance substance, double volume_in_mm)
{
    string new_name = substance.get_name();
    int substance_pos_in_cup = get_substance_id(substances, new_name, false);

    if (substance_pos_in_cup == -1)
    {
        substances.push_back(substance);
        volumes.push_back(volume_in_mm / 1e6);
    }
    else {
        volumes[substance_pos_in_cup] += volume_in_mm / 1e6;
    }

}
```

Konsola debugowania programu Microsoft Visual Studio

```
Nie znaleziono plynu o podanej nazwie: kwas!



woda; volume: 250ml; mass: 249.45g
mleko; volume: 50ml; mass: 51.5g

Volume coe: 0.833333, 0.166667,
Mass coe: 0.828875, 0.171125,
```

# Zadanie kończące projekt

Zadanie:

- zaimplementować funkcję dodawania cieczy do uzyskania jej zadanego procentu objętościowego w kubku,
- zaimplementować funkcję dodawania cieczy do uzyskania jej zadanego procentu masowego w kubku,
- zaimplementować funkcjonalność zlewania zawartości dwóch kubków do jednego